Thesis Notebook

2D Orbit with the suns density profile and mass function, along with dissipative forces

Setting functions up and choosing variables

In[1]:=
```
(*R values in Schwarzschild coordinates*)
RStar = 8;(*Radius of the Star*)
M = 1;
```

Exterior Schwarzschild Isotropic coordinates definitions $r(\bar{r})$, $\bar{r}(r)$, $A(\bar{r})$, $\alpha(\bar{r})$

$r(\bar{r})$, $\bar{r}(r)$

In[3]:=
```
rofrbarOutside[rbar_] := rbar (1 + M / (2 rbar))^2;

rbarofrOutside[r_] := 1/2 (-M + r + Sqrt[r^2 - 2 M r]);
```

Shift, Lapse and rbarStar

In[5]:=
```
A[rbar_] := (1 + M / (2 rbar))^2;
α[rbar_] := (1 - M / (2 rbar)) / (1 + M / (2 rbar));
```

In[7]:=
```
rbarStar = rbarofrOutside[RStar] // N
```

Out[7]= 6.9641

Mass and Density functions from 2405.08113

Fixing the rho0 by letting the total mass integral equal to 1

In[8]:=
```
rho[r_] := rho0 (1 - r/R)^6;
(*massIntegral =  4 Pi Integrate[rho[r] r^2, {r, 0, R}]; Set equal to 1*)
```

In[9]:=
```
rhoCentre[R_] := 63/(Pi R^3) (*From the above*)
rho0 = rhoCentre[RStar];
```

Mass function

In[11]:=
```
MassFunc[r_] :=
  Piecewise[{{252 (r^9/(9 RStar^9) - 3 r^8/(4 RStar^8) + 15 r^7/(7 RStar^7) - 10 r^6/(3 RStar^6) + 3 r^5/RStar^5 - 3 r^4/(2 RStar^4) + r^3/(3 RStar^3)),
     r ≤ RStar},
    {M, r > RStar}
   }];
(*In 2405.08113 this function is divided by the solar mass, here I use M=1*)
```

Density function

In[12]:=
```
rho[r_] := Piecewise[{
    {rho0 (1 - r/RStar)^6, r < RStar},
    {0, r ≥ RStar}
   }];
```

Numerically integrating for the pressure and the phi in α = exp(phi)

Defining the ODEs

In[13]:=
```
(*TOV differential equation*)
dPdr[r_, Pressure_] := -(rho[r] + Pressure) ((MassFunc[r] + 4 Pi Pressure r^3)/(r (r - 2 MassFunc[r])));
```

In[14]:=
```
(*Phi differential equation for metric*)
dPhidrMetric[r_, Pressure_] := ((MassFunc[r] + 4 Pi Pressure r^3)/(r (r - 2 MassFunc[r])));
```

Numerically integrating these coupled ODEs

In[15]:=
```
rmin = 10^-16 (*For these integrations*);
```

In[16]:=
```
eq1 = P'[r] == dPdr[r, P[r]];
eq2 = Phi'[r] == dPhidrMetric[r, P[r]];
BCs = {P[RStar] == 0, Phi[RStar] == 1/2 Log[1 - 2 M/RStar]};
(*Phi[RStar] from the Schwarzschild metric*)
```

In[19]:=
```
sol1 = NDSolve[
    {eq1,
     eq2,
     BCs},
    {P, Phi},
    {r, RStar, rmin}
   ][[1]]; (*Very small rmin to avoid singularity sarp mentioned*)
```

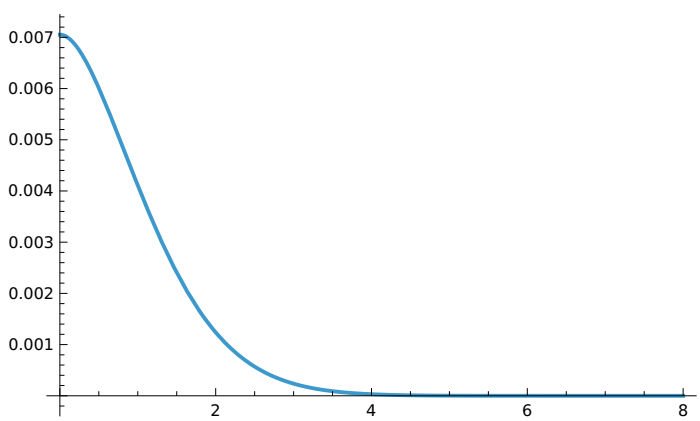
In[20]:=
```
Pressure[r_] := P[r] /. sol1

PhiMetric[r_] = Phi[r] /. sol1;

(*Plot[Derivative[1][PhiMetric][r], {r, 0, RStar}]*)
(*Causing the issue it seems, but we have this Analytically!*)
```

In[22]:=
```
Plot[Pressure[r], {r, 0, RStar}, PlotRange → All]
```

Out[22]=



Solving for $\bar{r}(r)$ and $r(\bar{r})$ inside

In[23]:=
```
MassFuncdummy[r_] :=
```

$$252\left(\frac{r^9}{9\,\text{RStar}^9}-\frac{3\,r^8}{4\,\text{RStar}^8}+\frac{15\,r^7}{7\,\text{RStar}^7}-\frac{10\,r^6}{3\,\text{RStar}^6}+\frac{3\,r^5}{\text{RStar}^5}-\frac{3\,r^4}{2\,\text{RStar}^4}+\frac{r^3}{3\,\text{RStar}^3}\right)$$

In[24]:=

$$\text{drbardr}[\text{rbar_},\ \text{r_}] := \frac{\text{rbar}}{\text{r}}\left(1-\frac{2\,\text{MassFuncdummy}[\text{r}]}{\text{r}}\right)^{-\frac{1}{2}}$$

Numerically Integrating the above equation

In[25]:=
```
(*Mismatch in derivatives due to accuracy of this integral*)
(*Turns out it makes no difference*)
```

In[26]:=
```
(*RStar=SetPrecision[8,40];
rmin=SetPrecision[10^-16,40];
rbarStarr = SetPrecision[rbarStar, 40];*)
```

In[27]:=
```
eq = rbarofr'[r] == drbardr[rbarofr[r], r];
BC = {rbarofr[RStar] == rbarStar};
```

In[29]:=
```
sol2 = NDSolve[
    {eq,
     BC},
    {rbarofr},
    {r, RStar, rmin}
    (*WorkingPrecision→40,
    AccuracyGoal→20, (*I want the answer correct up to 10^-8 → 8 correct decimals*)
    PrecisionGoal→20*)
    (*I want 8 correct digits relative to the size of the number. E.g if
     answer is 1.234567899, then precision goal gives me say 1.23456789 -
     the amount of digits is 8 and accuracy goal makes these 8 digits exact.*)
   ][[1]];
```

In[30]:=
```
rbarofrInside[r_] = rbarofr[r] /. sol2;
```

Some sanity checks

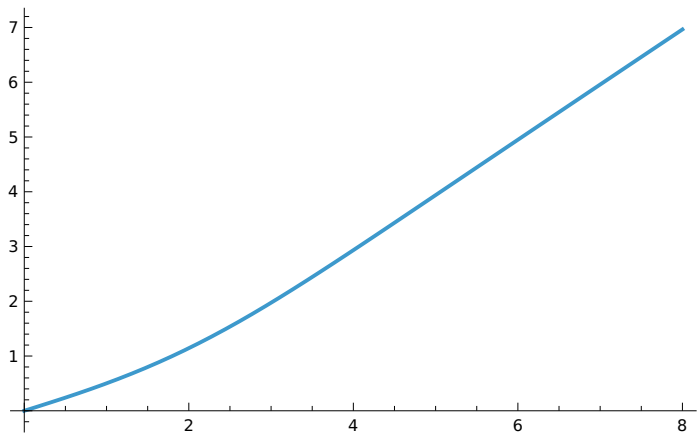
In[31]:=
```
rbarofrInside[RStar]
rbarofrOutside[RStar] // N
```

Out[31]=
```
6.9641
```

Out[32]=
```
6.9641
```

In[33]:=
```
Plot[rbarofrInside[r], {r, 0, RStar}]
```

Out[33]=



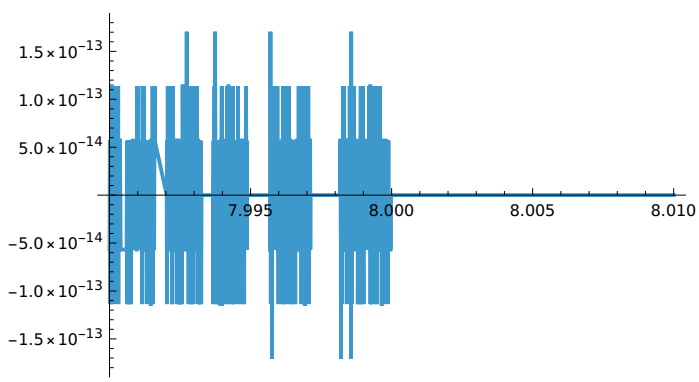
In[34]:=
```
δ = .5
```

Out[34]=
```
0.5
```

In[35]:=
```
δ = .01
Plot[Derivative[1][MassFunc][r], {r, RStar - δ, RStar + δ}]
```

Out[35]=
```
0.01
```

Out[36]=



Now using the interpolation trick to obtain $r(\bar{r})$:

In[37]:=
```
MassFuncdummy[r_] :=
```

$$252\left(\frac{r^9}{9\,\text{RStar}^9}-\frac{3\,r^8}{4\,\text{RStar}^8}+\frac{15\,r^7}{7\,\text{RStar}^7}-\frac{10\,r^6}{3\,\text{RStar}^6}+\frac{3\,r^5}{\text{RStar}^5}-\frac{3\,r^4}{2\,\text{RStar}^4}+\frac{r^3}{3\,\text{RStar}^3}\right)$$

In[38]:=

$$\text{drdrbar}[r_,\ \text{rbar}_] := \frac{r}{\text{rbar}}\left(1-\frac{2\,\text{MassFuncdummy}[r]}{r}\right)^{\frac{1}{2}}$$

In[39]:=
```
rmin = 10^-16;
rbarStar
```

Out[40]=
```
6.9641
```

In[41]:=
```
eq = rofrbar'[rbar] == drdrbar[rofrbar[rbar], rbar];
BC = {rofrbar[rbarStar] == RStar};
```

In[43]:=
```
sol3 = NDSolve[
    {eq, BC},
    rofrbar,
    {rbar, rbarStar, rmin}
   ][[1]];
```

In[44]:=
```
rofrbarInside[rbar_] = rofrbar[rbar] /. sol3;
```

In[45]:=
```
(*Table[ expression, {variable, start, end, step}
    data = Prepend[Table[
     {rbarofrInside[r], r},
     {r, rmin, RStar, (RStar - rmin)/10000} (*10000 points and A is not messy*)
    ], {0,0}];*)
```

In[46]:=
```
(*rofrbarInside = Interpolation[data];*)
```

In[47]:=
```
rofrbarFull[rbar_] := Piecewise[
  {
   {rofrbarOutside[rbar], rbar ≥ rbarStar},
   {rofrbarInside[rbar], rbar < rbarStar}
  }
 ]
```

In[48]:=
```
rbarofrFull[r_] := Piecewise[
  {
   {rbarofrOutside[r], r ≥ RStar},
   {rbarofrInside[r], r < RStar}
  }
 ]
```

In[49]:=
```
Plot[rofrbarFull[rbar], {rbar, rbar0 - δ, rbar0 + δ}]
```

Plot: Limiting value −0.01 + rbar0 in {rbar, −0.01 + rbar0, 0.01 + rbar0} is not a machine-sized real number.

Out[49]=
```
Plot[rofrbarFull[rbar], {rbar, rbar0 - δ, rbar0 + δ}]
```

Finding the interior metric functions

Getting αInside($\bar{r}$):

In[50]:=
```
αInside[rbar_?NumericQ] := Exp[PhiMetric[rofrbarFull[rbar]]]
```

Sanity checks

Making the derivative of alpha continuous

Interior derivative in terms of r

Doing things in r first as this is more analytic

In[56]:=
```
dPhidrMetric[r_] := (MassFunc[r] + 4 Pi Pressure[r] r^3)/(r (r - 2 MassFunc[r]))
```

In[57]:=
```
alpha[r_] := Exp[PhiMetric[r]]
DERalphaIns[r_] := dPhidrMetric[r] × alpha[r] (*Analytic times numerical integral*)
```

Exterior derivative

In[59]:=
```
DERαOutsiderbardummy = Derivative[1][α][rbar];
DERαOutsiderbar[r_] := (1 - 1/(2 rbar))/(2 (1 + 1/(2 rbar))^2 rbar^2) + 1/(2 (1 + 1/(2 rbar)) rbar^2) /. rbar → rbarofrOutside[r]
(*More accurate subbing in after for rbar maybe?*)
drbardr[r_] := rbarofrOutside[r]/r (1 - (2 MassFunc[r])/r)^(-1/2)
```

In[62]:=
```
DERalpha[r_] := drbardr[r] × DERαOutsiderbar[r] (*Chain rule*)
```

In[63]:=
```
rofrbarOutside[rbarStar]
```

Out[63]=

8.

In[64]:=
```
rbarofrOutside[rofrbarOutside[rbarStar]] // N
```

Out[64]=

6.9641

Checking to see if it worked

Numbers at star boundary

In[65]:=
```
DERalpha[RStar] // N
DERalphaIns[RStar] // N
```

Out[65]=
```
0.0180422
```

Out[66]=
```
0.0180422
```

Proper definitions in isotropic coords now

In[67]:=
```
DERαOutside[rbar_] := Module[{r = rofrbarOutside[rbar]}, drbardr[r] × DERαOutsiderbar[r]]

DERαInside[rbar_] := Module[{r = rofrbarFull[rbar]},
  dPhidrMetric[r] × alpha[r]]
```

In[69]:=
```
DERαInside[rbarStar]
```

Out[69]=
```
0.0180422
```

In[70]:=
```
DERαOutside[rbarStar] // N
```

Out[70]=
```
0.0180422
```

In[71]:=
```
(*DERαFull[rbar_]:=If[rbar<rbarStar,DERαInside[rbar],DERαOutside[rbar]]*)
```

Going to do the same derivative setup as the α component for the A component

In[72]:=
```
AInside[rbar_?NumericQ] := rofrbarFull[rbar] / rbar;
```

In[73]:=
```
A[rbarStar]
AInside[rbarStar]
```

Out[73]=
```
1.14875
```

Out[74]=
```
1.14875
```

In[75]:=
```
drdrbar[rbar_] := rofrbarFull[rbar] / rbar (1 - 2 MassFunc[rofrbarFull[rbar]] / rofrbarFull[rbar])^(1/2)
```

In[76]:=
```
DERAInside[rbar_] := drdrbar[rbar] / rbar - rofrbarFull[rbar] / rbar^2;
DERAOutside[rbar_] := Derivative[1][A][rbar]
```

In[78]:=
```
DERAOutside[rbarStar]
DERAInside[rbarStar]
```

Out[78]=
```
-0.0220995
```

Out[79]=
```
-0.0220995
```

In[80]:=
```
AFull[rbar_] := Piecewise[
  {
   {A[rbar], rbar ≥ rbarStar},
   {AInside[rbar], rbar < rbarStar}
  }
 ]
```

In[81]:=
```
αFull[rbar_] := Piecewise[
  {
   {α[rbar], rbar ≥ rbarStar},
   {αInside[rbar], rbar < rbarStar}
  }
 ]
```

In[82]:=
```
Plot[AFull[rbar], {rbar, rbar0 - 0.000001, rbar0 + .000001}]
```

Plot: Limiting value $-1.\times 10^{-6}+\text{rbar0}$ in $\{\text{rbar}, -1.\times 10^{-6}+\text{rbar0}, 1.\times 10^{-6}+\text{rbar0}\}$ is not a machine-sized real number.

Out[82]=
```
Plot[AFull[rbar], {rbar, rbar0 - 1. × 10^-6, rbar0 + 1. × 10^-6}]
```

In[83]:=
```
Plot[αFull[rbar], {rbar, rbar0 - 0.00000001, rbar0 + .00005}]
```

Plot: Limiting value $-1.\times 10^{-8}+\text{rbar0}$ in $\{\text{rbar}, -1.\times 10^{-8}+\text{rbar0}, 0.00005+\text{rbar0}\}$ is not a machine-sized real number.

Out[83]=
```
Plot[αFull[rbar], {rbar, rbar0 - 1. × 10^-8, rbar0 + 0.00005}]
```

Density and Pressure in isotropic coordinates

In[84]:=
```
rho0 = rhoCentre[RStar];
```

In[85]:=
```
rho[rbar_] := Piecewise[
  {
   {rho0 (1 - rofrbarFull[rbar]/RStar)^6, rbar < rbarStar},
   {0, rbar ≥ rbarStar}
  }
 ];
```

In[86]:=
```
rho[rbar_] := Piecewise[
  {
   {.5, rbar < rbarStar},
   {0.5, rbar ≥ rbarStar}
  }
 ];
```

In[87]:=
```
(*Maybe issue with NDSolve is that there are two numerically
 integrated things here being used and its causing issues?*)
(*Going to make the above as one interpolating function*)
data = Table[{rbarofrInside[r], Pressure[r]},
  {r, 0, RStar, RStar/1000}];
```

In[88]:=

$$\text{IsotropicCoordsPressure[rbar_] :=}\quad \text{rho[rbar]}\left(\frac{\sqrt{1-2\frac{M r^2}{\text{RStar}^3}}-\sqrt{1-2\frac{M}{\text{RStar}}}}{3\sqrt{1-2\frac{M}{\text{RStar}}}-\sqrt{1-2\frac{M r^2}{\text{RStar}^3}}}\right) /.\ r \to \text{rofrbarFull[rbar]};$$

Sorting out the speed of sound as a function of r: a(r)

Speed of sound in isotropic coordinates

In[89]:=
```
asosPiecewise[rbar_?NumericQ] := Piecewise[{
  {.5, rbar < rbarStar},
  {.5, rbar ≥ rbarStar}
 }
]
```

Setting up 2D ODEs for new density profile:

Constants that are going to be used

In[90]:=
```
m = 10^-6;
```

Mach number interpolation

Interpolating

In[91]:=
```
fMachLow[m_] := 1/2 Log[(1 + m)/(1 - m)] - m ;
fMachhigh[m_] := 1/2 Log[1 - 1/m^2] + 10
```

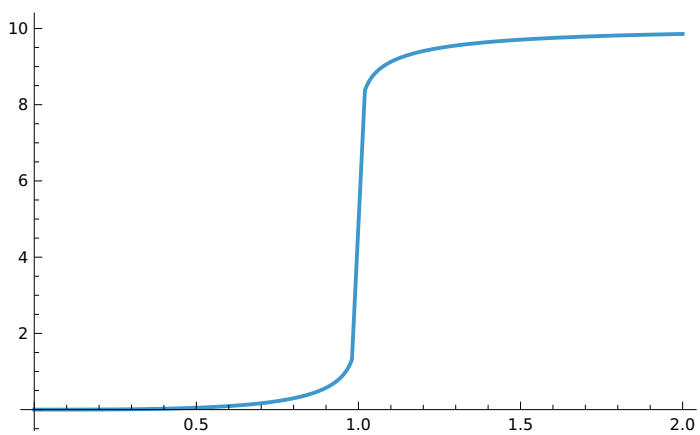
In[93]:=
```
x1 = .98;
x2 = 1.02;
y1 = fMachLow[x1];
y2 = fMachhigh[x2];
```

In[97]:=
```
anaylticline[Mach_] := y1 + (y2 - y1)(Mach - x1)/(x2 - x1)
```

In[98]:=
```
IVariableInterpolated[Mach_] := Piecewise[{
    {1/2 Log[(1 + Mach)/(1 - Mach)] - Mach, Mach < 0.98},
    {anaylticline[Mach], 0.98 < Mach < 1.02},
    {1/2 Log[1 - 1/Mach^2] + 10, Mach > 1.02}
  }];
```

In[99]:=
```
Plot[IVariableInterpolated[Mach], {Mach, 0, 2}, Exclusions → None, PlotRange → All]
```

Out[99]=



Outside Star

Velocities

In[100]:=

$$\text{Ut}[\text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{\sqrt{1 + \left(\frac{\text{ur}}{\text{A[rbar]}}\right)^2 + \left(\frac{\text{uphi}}{\text{rbar A[rbar]}}\right)^2}}{\alpha[\text{rbar}]};$$

$$\text{umagOutside}[\text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{\sqrt{\left(\text{ur}^2 + \left(\frac{\text{uphi}}{\text{rbar}}\right)^2\right)}}{\text{A[rbar]}}$$

Rates of change of velocities

In[102]:=

$$\text{drBardt}[\text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{1}{\text{A[rbar]}^2\, \text{Ut}[\text{rbar}, \text{ur}, \text{uphi}]}\, \text{ur};$$

$$\text{dphidtOutside}[\text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{1}{\text{rbar}^2 * \text{A[rbar]}^2 * \text{Ut}[\text{rbar}, \text{ur}, \text{uphi}]}\, \text{uphi};$$

Gravitational Waves

In[104]:=

```
FRRr[m_, rbar_, ur_, uphi_] :=
  Module[{r = rofrbarFull[rbar]}, ((8 m^2)/(15 r^4)*drBardt[rbar, ur, uphi]*
     (8 M^2 + r M (6 drBardt[rbar, ur, uphi]^2 + 36 r^2 dphidtOutside[rbar, ur, uphi]^2)))];
```

In[105]:=

```
FRRphi[m_, rbar_, ur_, uphi_] :=
  Module[{r = rofrbarFull[rbar]}, ((8 m^2)/(5 r^4)*dphidtOutside[rbar, ur, uphi]*
     (2 M^2 + r M (9 drBardt[rbar, ur, uphi]^2 - 6 r^2 dphidtOutside[rbar, ur, uphi]^2)))];
```

In[106]:=

$$\text{duphidtOutside}[\text{m_}, \text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{\eta}{\text{m}} \left(\text{A[rbar]}^2\, \text{rbar}^2\, \text{FRRphi}[\text{m}, \text{rbar}, \text{ur}, \text{uphi}]\right);$$

$$\text{durdt2DOutside}[\text{m_}, \text{rbar_}, \text{ur_}, \text{uphi_}] :=$$
$$-\alpha[\text{rbar}] \times \text{Ut}[\text{rbar}, \text{ur}, \text{uphi}] \times \text{DER}\alpha\text{Outside}[\text{rbar}] + \frac{\text{ur}^2}{\text{Ut}[\text{rbar}, \text{ur}, \text{uphi}]} \left(\frac{\text{DERAOutside}[\text{rbar}]}{\text{A[rbar]}^3}\right) +$$
$$\frac{\text{uphi}^2}{\text{Ut}[\text{rbar}, \text{ur}, \text{uphi}]} \left(\frac{1}{\text{rbar}^3 * \text{A[rbar]}^2} + \frac{\text{DERAOutside}[\text{rbar}]}{\text{rbar}^2 * \text{A[rbar]}^3}\right) +$$
$$\frac{\eta}{\text{m}} \left(\text{A[rbar]}^2\, \text{FRRr}[\text{m}, \text{rbar}, \text{ur}, \text{uphi}]\right);$$

Inside Star

Velocities

In[108]:=

$$\text{UtInside}[\text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{\sqrt{1 + \left(\frac{\text{ur}}{\text{AInside}[\text{rbar}]}\right)^2 + \left(\frac{\text{uphi}}{\text{rbar AInside}[\text{rbar}]}\right)^2}}{\alpha\text{Inside}[\text{rbar}]};$$

$$\text{umagInside}[\text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{\sqrt{\left(\text{ur}^2 + \left(\frac{\text{uphi}}{\text{rbar}}\right)^2\right)}}{\text{AInside}[\text{rbar}]}$$

Mach business

In[110]:=

$$\text{Mach}[\text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{\frac{\sqrt{\left(\text{ur}^2+\left(\frac{\text{uphi}}{\text{rbar}}\right)^2\right)}}{\text{AInside}[\text{rbar}]}}{\text{asosPiecewise}[\text{rbar}] \times \alpha\text{Inside}[\text{rbar}] \times \text{UtInside}[\text{rbar}, \text{ur}, \text{uphi}]}$$

Dissipative Forces

Accretion

In[111]:=

$$\text{mdot}[\text{m_}, \text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{4\,\text{Pi rho}[\text{rbar}]\,\text{m}^2}{\left(\text{asosPiecewise}[\text{rbar}]^2 + \left(\frac{\text{umagInside}[\text{rbar}, \text{ur}, \text{uphi}]}{\alpha\text{Inside}[\text{rbar}] \times \text{UtInside}[\text{rbar}, \text{ur}, \text{uphi}]}\right)^2\right)^{\frac{3}{2}}};$$

```
(*General Bondi-Hoyle-Lyttleton formula for mass accretion with q = 3 and λ=1*)
```

In[112]:=

```
dpdtaccretionr[m_, rbar_, ur_, uphi_] :=
  -((4 Pi αInside[rbar] (rho[rbar] + IsotropicCoordsPressure[rbar]) m^2))
   (1/((asosPiecewise[rbar]^2 + (umagInside[rbar, ur, uphi]/(αInside[rbar] UtInside[rbar, ur, uphi]))^2)^(3/2)) ) ur;

dpdtaccretionphi[m_, rbar_, ur_, uphi_] :=
 -((4 Pi αInside[rbar] (rho[rbar] + IsotropicCoordsPressure[rbar]) m^2))
   (1/((asosPiecewise[rbar]^2 + (umagInside[rbar, ur, uphi]/(αInside[rbar] UtInside[rbar, ur, uphi]))^2)^(3/2)) ) uphi
```

Drag

In[114]:=

```
dpdtdragphi[m_, rbar_, ur_, uphi_] := 1/umagInside[rbar, ur, uphi]^3 (-4 π αInside[rbar]*
    (rho[rbar] + IsotropicCoordsPressure[rbar])*m^2*(1 + 2 umagInside[rbar, ur, uphi]^2)^2 *
    IVariableInterpolated[Mach[rbar, ur, uphi]] *uphi);

dpdtdragr[m_, rbar_, ur_, uphi_] :=
  (-4 π αInside[rbar] (rho[rbar] + IsotropicCoordsPressure[rbar]) m^2
    (1 + 2 umagInside[rbar, ur, uphi]^2)^2 IVariableInterpolated[Mach[rbar, ur, uphi]] *
    ur)/umagInside[rbar, ur, uphi]^3;
```

Rates of change of velocities

In[116]:=

```
drBardtInside[rbar_, ur_, uphi_] := 1/(AInside[rbar]^2 UtInside[rbar, ur, uphi]) ur;

dphidtInside[rbar_, ur_, uphi_] := 1/(rbar^2 AInside[rbar]^2 UtInside[rbar, ur, uphi]) uphi;
```

In[118]:=

```
(*Redefining the mass function without the piecewise component for simplicity*)
MassFunction[r_] :=
```

$$252\left(\frac{r^9}{9\,\text{RStar}^9}-\frac{3\,r^8}{4\,\text{RStar}^8}+\frac{15\,r^7}{7\,\text{RStar}^7}-\frac{10\,r^6}{3\,\text{RStar}^6}+\frac{3\,r^5}{\text{RStar}^5}-\frac{3\,r^4}{2\,\text{RStar}^4}+\frac{r^3}{3\,\text{RStar}^3}\right);$$

In[119]:=

```
Mp[r_] := Derivative[1][MassFunction][r]
Mpp[r_] := Derivative[2][MassFunction][r]
Mppp[r_] := Derivative[3][MassFunction][r]
```

In[122]:=

```
FRRrIns[m_, rbar_, ur_, uphi_] :=
  Module[{r = rofrbarFull[rbar]}, (8 m^2)/(15 r^4)*drBardtInside[rbar, ur, uphi]*
    (8 MassFunction[r]^2 + r MassFunction[r] (6 drBardtInside[rbar, ur, uphi]^2 +
          36 r^2 dphidtInside[rbar, ur, uphi]^2 + Mp[r] - 3 r Mpp[r]) -
       r^2 (Mp[r]^2 + 3 r^2 dphidtInside[rbar, ur, uphi]^2 (6 Mp[r] - r Mpp[r]) +
          drBardtInside[rbar, ur, uphi]^2 (2 Mp[r] + r Mpp[r] - r^2 Mppp[r])))];
```

In[123]:=

```
FRRphiIns[m_, rbar_, ur_, uphi_] :=
  Module[{r = rofrbarFull[rbar]}, (8 m^2)/(5 r^4)*dphidtInside[rbar, ur, uphi]*
    (2 MassFunction[r]^2 + 2 r^4 dphidtInside[rbar, ur, uphi]^2 × Mp[r] + r MassFunction[r]
        (9 drBardtInside[rbar, ur, uphi]^2 - 6 r^2 dphidtInside[rbar, ur, uphi]^2 - 2 Mp[r]) -
       r^2 drBardtInside[rbar, ur, uphi]^2 (7 Mp[r] - 2 r Mpp[r]))];
```

In[124]:=

```
duphidtInside[m_, rbar_, ur_, uphi_] :=
  η/m (dpdtaccretionphi[m, rbar, ur, uphi] + dpdtdragphi[m, rbar, ur, uphi] +
      AInside[rbar]^2 rbar^2 FRRphiIns[m, rbar, ur, uphi]);

durdt2DInside[m_, rbar_, ur_, uphi_] :=
  -αInside[rbar] × UtInside[rbar, ur, uphi] × DERαInside[rbar] +
    ur^2/UtInside[rbar, ur, uphi] (DERAInside[rbar]/AInside[rbar]^3) +
    uphi^2/UtInside[rbar, ur, uphi] (1/(rbar^3 AInside[rbar]^2) + DERAInside[rbar]/(rbar^2 AInside[rbar]^3)) +
    η/m (dpdtaccretionr[m, rbar, ur, uphi] + dpdtdragr[m, rbar, ur, uphi] +
        AInside[rbar]^2 FRRrIns[m, rbar, ur, uphi]);
```

In[126]:=

```
(* + 0*AInside[rbar]^2 FRRrIns[rbar, ur, uphi]
  + 0*AInside[rbar]^2 rbar^2 FRRphiIns[rbar, ur, uphi]*)
```

Defining Energy and Angular momentum per unit mass functions along with other orbital parameters

In[127]:=

```
EnergyPerUnitMass[ptilde_, eccen_] := Sqrt[((ptilde - 2)^2 - 4 * eccen^2)/(ptilde * (ptilde - 3 - eccen^2))];

LPerUnitMass[ptilde_, M_, eccen_] := (ptilde * M)/Sqrt[ptilde - 3 - eccen^2];

ptildefunc[p_, Mass_] := p/Mass;

rapoapsis[eccen_, ptilde_, Mass_] := ((ptilde) * Mass)/(1 - eccen);

(*In Schwarzschild areal r coordinates*)
rperiapsis[eccen_, ptilde_] := ptilde/(1 + eccen);
```

Defining the constants to define the motion

Eccentricity and semi-latus rectum

In[132]:=

```
eccentricity = 0.4;
(*eccentricity = 9/11; (*Obtained from setting periapsis to 5.5 and ptilde still to 10*)*)
semilatrecP = 10;
ptilde = ptildefunc[semilatrecP, M];
```

Energy and Angular momentum per unit mass

In[135]:=

```
Energy = EnergyPerUnitMass[ptilde, eccentricity];
AngularMomentumUphi0 = LPerUnitMass[ptilde, M, eccentricity];
```

Initial drop radius (apoapsis), closest point it goes to (perihelion), defining Isotropic star radius

In[137]:=

```
r0 = rapoapsis[eccentricity, ptilde, M];
rbar0 = rbarofrOutside[r0];

rperiapsis[eccentricity, ptilde];
(*Setting it up such that it passes the point where I know the accretion is
 strong enough to sufficiently slow it down from the inside simulation.
  (Needed this for accretion and drag to work independently and together, GW allowed
     me to go back to it not having to pass through the star on the first orbit)
 Used this to find the proper rapoapsis*)
```

In[140]:=

```
(*r_a + r_p = 2a, a = p/(1-e^2) = 30.25
    r_a = 60.5-5.5 = 55 = rbar0 = r_apoapsis*)
```

Must satisfy this inequality to pass through star

In[141]:=

```
ptilde < RStar (1 + eccentricity)
ptilde > 6 + 2 eccentricity
```

Out[141]=

True

Out[142]=

True

Solving the ODEs

In[143]:=

```
drBardtInside[rbarStar, 0, AngularMomentumUphi0]
durdt2DInside[m, rbarStar, 0, AngularMomentumUphi0]
duphidtInside[m, rbarStar, 0, AngularMomentumUphi0]
dphidtInside[rbarStar, 0, AngularMomentumUphi0]
```

Out[143]=

0.

Out[144]=

0.00219943

Out[145]=

$-0.000250407\,\eta$

Out[146]=

0.0466817

In[223]:=

```
t0 = 0;
tEnd = 50 000;
η = 7.25 × 10^3;
```

```
solIn = NDSolve[
    {
     rbar'[t] ==
      Piecewise[
       {
        {drBardt[rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {drBardtInside[rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     ur'[t] ==
      Piecewise[
       {
        {durdt2DOutside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {durdt2DInside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     uphi'[t] ==
      Piecewise[
       {
        {duphidtOutside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {duphidtInside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     phi'[t] ==
      Piecewise[
       {
        {dphidtOutside[rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {dphidtInside[rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     mass'[t] ==
      Piecewise[
       {
        {0, rbar[t] ≥ rbarStar},
        {mdot[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     rbar[t0] == rbar0,
     ur[t0] == 0,
     phi[t0] == 0,
     uphi[t0] == AngularMomentumUphi0,
```

```
    mass[t0] == m,

    WhenEvent[rbar[t] ≤ 10^-2 , "StopIntegration"]
   },
   {rbar, ur, phi, uphi, mass},
   {t, t0, tEnd},
   MaxSteps → Infinity,
   WorkingPrecision → MachinePrecision
   (*Machine precision and 20k max steps also works*)
  ][[1]];
```

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

In[151]:=

```
rbarNumIn = rbar /. solIn;
urNumIn = ur /. solIn;
phiNumIn = phi /. solIn;
uphiNumIn = uphi /. solIn;
massNumIn = mass /. solIn;
```

In[156]:=

```
tFinish = rbarNumIn["Domain"][[1, 2]]
tSurface = t /. FindRoot[rbarNumIn[t] == rbarStar, {t, 10, 10, tFinish}]
```

Out[156]=

303.46859

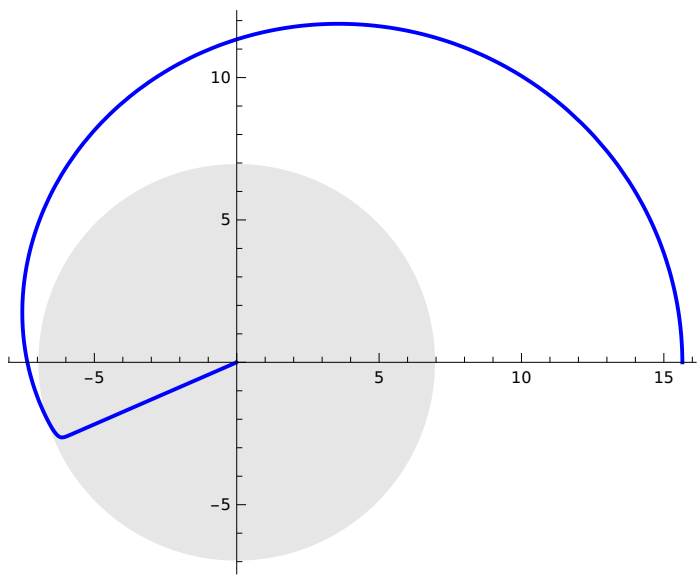
Out[157]=

160.095

In[158]:=

```
x[t_] := rbarNumIn[t] Cos[phiNumIn[t]];
y[t_] := rbarNumIn[t] Sin[phiNumIn[t]];
orbit2D = ParametricPlot[{x[t], y[t]}, {t, t0, 0.999 tFinish}, AspectRatio → Automatic,
   AxesLabel → {"x", "y"}, PlotRange → All, PlotStyle → {Thick, Blue}];
(*Automatic scale makes the aces the same scale*)
centralBody = Graphics[{GrayLevel[0.9], Disk[{0, 0}, rbarStar]}];
Show[centralBody, orbit2D, AspectRatio → Automatic, PlotRange → All, Axes → True]
```

Out[162]=



In[163]:=

```
FRRrFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {FRRr[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {FRRrIns[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[164]:=

```
Plot[FRRrFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 2500, tFinish}, PlotRange → All]
```

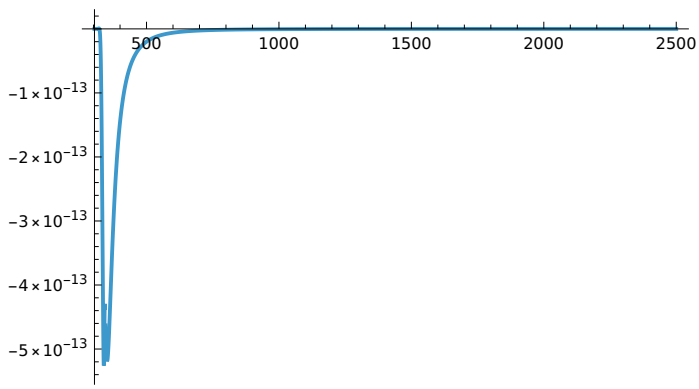
InterpolatingFunction: Input value {303.513} lies outside the range of data in the interpolating function. Extrapolation will be used.

InterpolatingFunction: Input value {303.513} lies outside the range of data in the interpolating function. Extrapolation will be used.

InterpolatingFunction: Input value {303.513} lies outside the range of data in the interpolating function. Extrapolation will be used.

General: Further output of InterpolatingFunction::dmval will be suppressed during this calculation.

Out[164]=



In[165]:=

```
FRRphiFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {FRRphi[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {FRRphiIns[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[166]:=

```
Plot[FRRphiFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 2500, tFinish}, PlotRange → All]
```

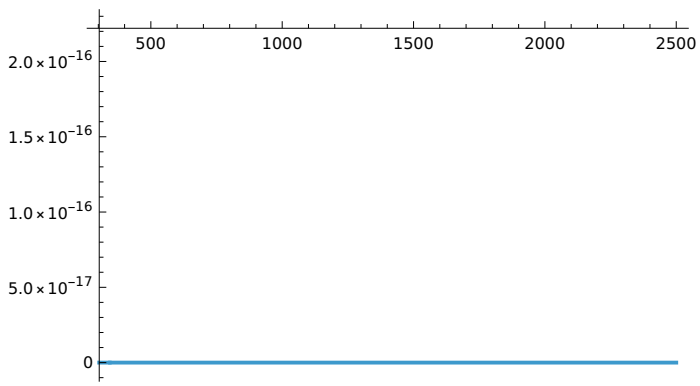
InterpolatingFunction: Input value {303.513} lies outside the range of data in the interpolating function. Extrapolation will be used.

InterpolatingFunction: Input value {303.513} lies outside the range of data in the interpolating function. Extrapolation will be used.

InterpolatingFunction: Input value {303.513} lies outside the range of data in the interpolating function. Extrapolation will be used.

General: Further output of InterpolatingFunction::dmval will be suppressed during this calculation.

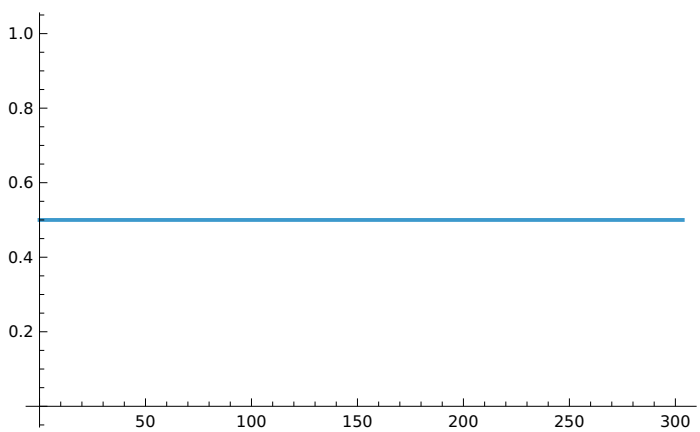
Out[166]=



In[167]:=

```
Plot[asosPiecewise[rbarNumIn[t]], {t, 0, tFinish}, PlotRange → All]
```

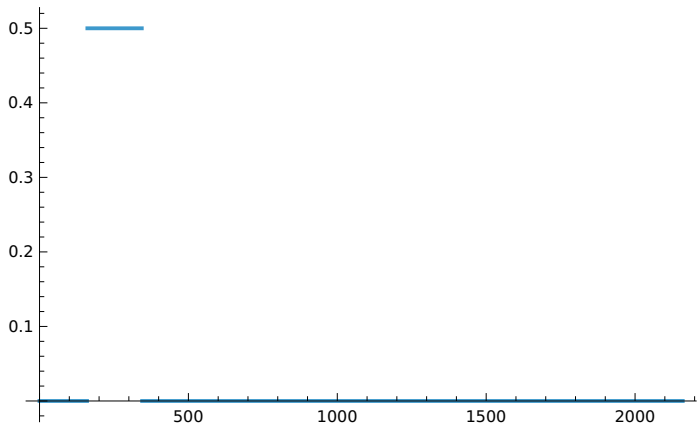
Out[167]=



In[168]:=

```
Plot[
 {
  Piecewise[
   {
    {0, rbarNumIn[t] ≥ rbarStar},
    {asosPiecewise[rbarNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

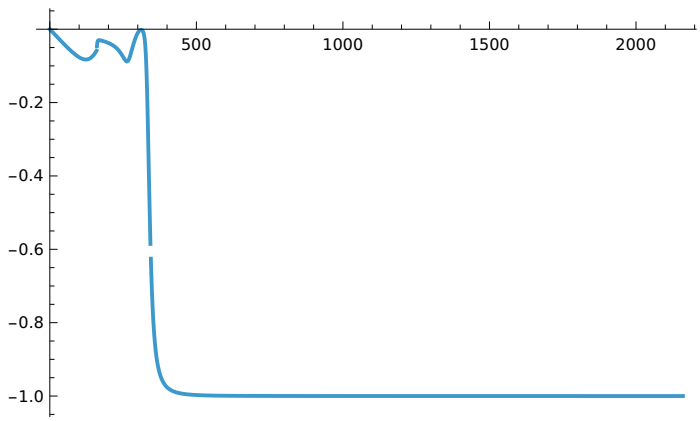
Out[168]=



In[169]:=

```
Plot[
 {
  Piecewise[
   {
    {drBardt[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {drBardtInside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

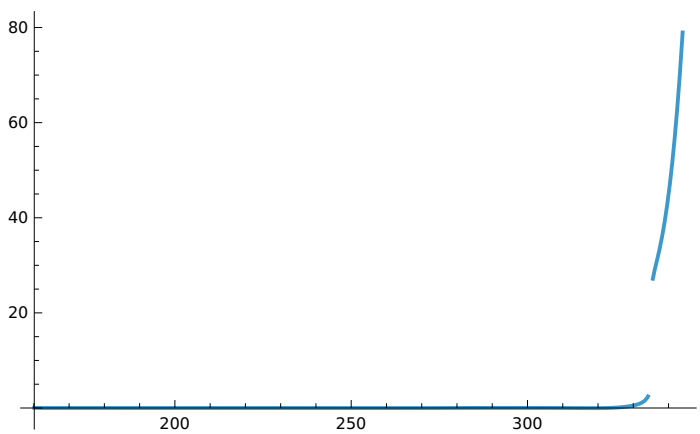
Out[169]=



In[170]:=

```
Plot[
 {
  Piecewise[
   {
    {durdt2DOutside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {durdt2DInside[massNumIn[t],
       rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

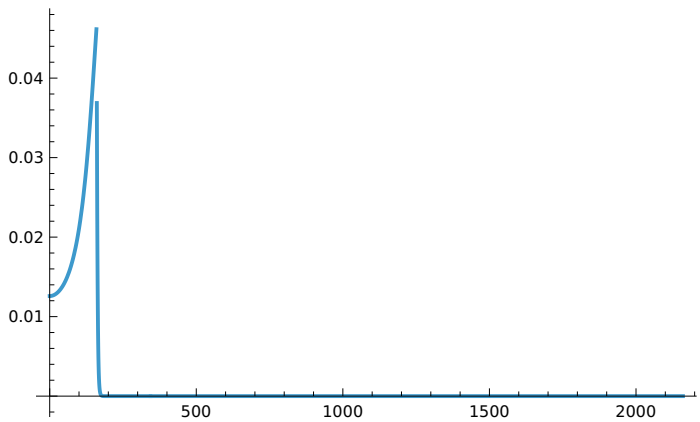
Out[170]=



In[171]:=

```
Plot[
 {
  Piecewise[
   {
    {dphidtOutside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {dphidtInside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

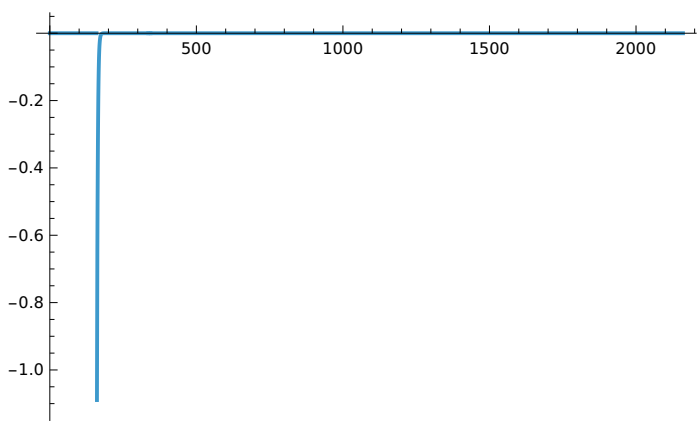
Out[171]=



In[172]:=

```
Plot[
 {
  Piecewise[
   {
    {duphidtOutside[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
     rbarNumIn[t] ≥ rbarStar},
    {duphidtInside[massNumIn[t],
       rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

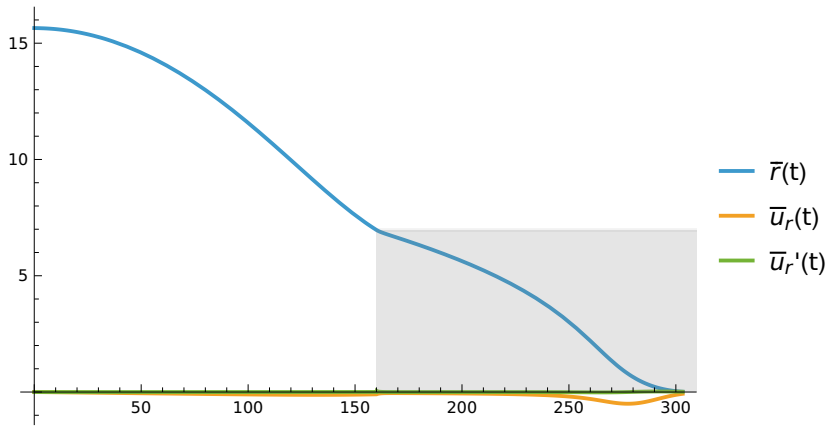
Out[172]=



In[173]:=

```
outFigRadial = Show[Plot[{rbarNumIn[t], urNumIn[t], urNumIn'[t]},
    {t, t0, .999 tFinish}, PlotLegends → LineLegend[Automatic, {"r̄(t)", "ū_r(t)", "ū_r'(t)"}]],
  Plot[rbarStar, {t, tSurface, tSurface + tFinish},
    PlotStyle → Directive[Gray, Opacity[0.1]], Filling → Bottom]]
```

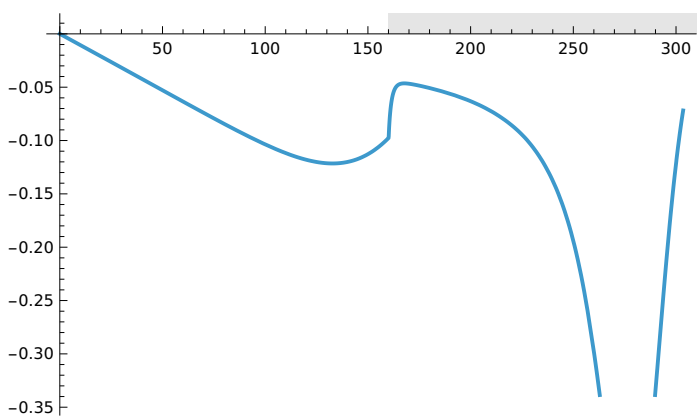
Out[173]=



In[174]:=

```
outFigRadial = Show[Plot[{urNumIn[t]}, {t, t0, tFinish},
    PlotLegends → LineLegend[Automatic, {"r̄(t)", "ū_r(t)", "ū_r'(t)"}]],
  Plot[rbarStar, {t, tSurface, tSurface + tFinish},
    PlotStyle → Directive[Gray, Opacity[0.1]], Filling → Bottom]]
```

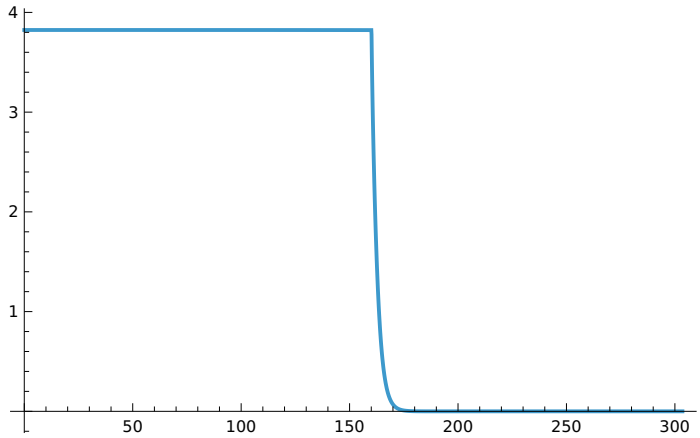
Out[174]=



In[175]:=

```
outFigAngular = Plot[{uphiNumIn[t]}, {t, t0, tFinish},
  PlotLegends → LineLegend[Automatic, {"phi(t)", "uphi(t)", "uphi'(t)"}]]
```

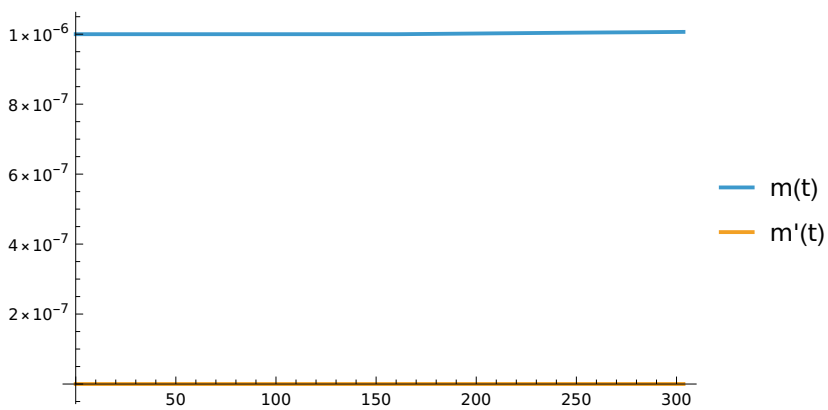
Out[175]=



In[176]:=

```
outFigMass = Show[Plot[{massNumIn[t], massNumIn'[t]},
   {t, t0, tFinish}, PlotLegends → LineLegend[Automatic, {"m(t)", "m'(t)"}]]]
```

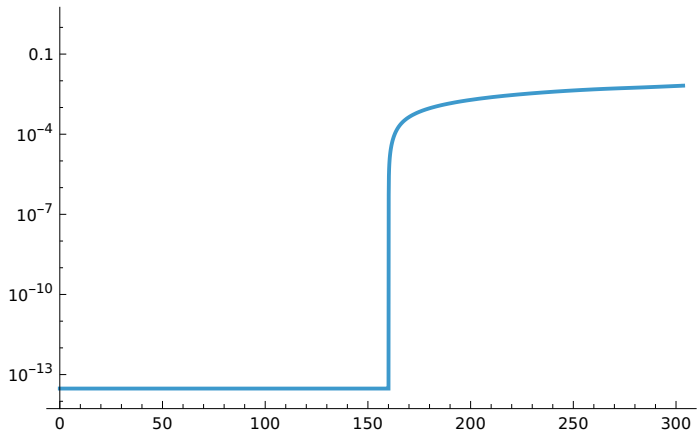
Out[176]=



In[177]:=

```
LogPlot[Abs[1 - massNumIn[t]/massNumIn[0]], {t, 0, tFinish}]
```

Out[177]=



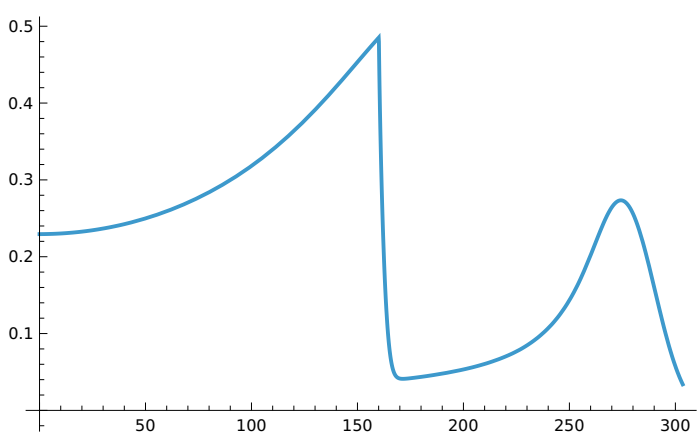
In[178]:=

```
umag[rbar_, ur_, uphi_] := Piecewise[
  {
   {umagOutside[rbar, ur, uphi], rbar ≥ rbarStar},
   {umagInside[rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[179]:=

```
Plot[umag[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], {t, 0, tFinish}, Exclusions → None]
(*Normal behaviour judging from rho=const simulation*)
```

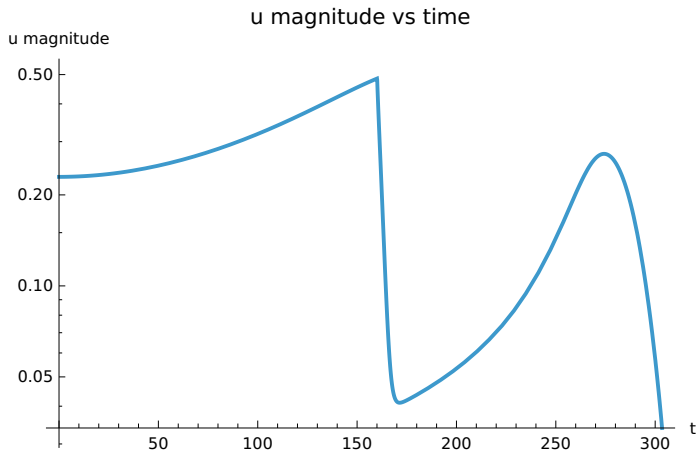
Out[179]=



In[180]:=

```
LogPlot[Evaluate[umag[rbarNumIn[t], urNumIn[t], uphiNumIn[t]] /. solIn], {t, 0, tFinish},
 PlotRange → All, AxesLabel → {"t", "u magnitude"}, PlotLabel → "u magnitude vs time"]
```

Out[180]=



In[181]:=

```
(*Show me the value of u at tfinish*)
umag[rbarNumIn[tFinish], urNumIn[tFinish], uphiNumIn[tFinish]] /. solIn
```

Out[181]=

0.0338824

Gravitational-wave strains

h_+

In[182]:=

```
hplusInside[d_, m_, rbar_, ur_, uphi_, phi_] := Module[{r = rofrbarFull[rbar], rd, phid},
  rd = drBardtInside[rbar, ur, uphi];
  phid = dphidtInside[rbar, ur, uphi];
  (-2 m/d) (MassFunc[r] Cos[2 phi]/r - rd^2 Cos[2 phi] + 2 rd r phid Sin[2 phi] + (r phid)^2 Cos[2 phi])
 ]
```

In[183]:=

```
hplusOutside[d_, m_, rbar_, ur_, uphi_, phi_] := Module[{r = rofrbarFull[rbar], rd, phid},
  rd = drBardt[rbar, ur, uphi];
  phid = dphidtOutside[rbar, ur, uphi];
  (-2 m/d) (M Cos[2 phi]/r - rd^2 Cos[2 phi] + 2 rd r phid Sin[2 phi] + (r phid)^2 Cos[2 phi])
 ]
```

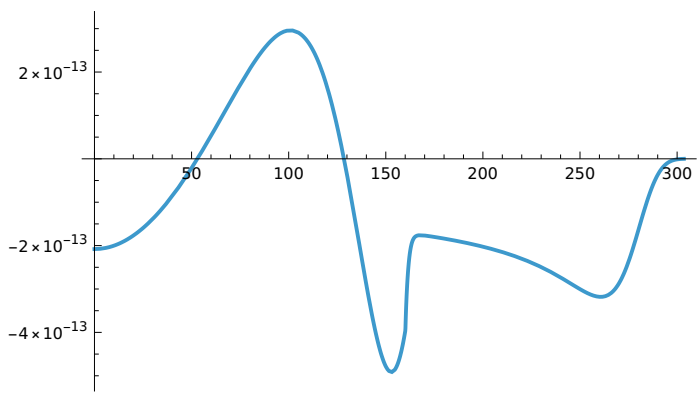
In[184]:=

```
hplusFull[d_, m_, rbar_, ur_, uphi_, phi_] := Piecewise[
  {
   {hplusOutside[d, m, rbar, ur, uphi, phi], rbar ≥ rbarStar},
   {hplusInside[d, m, rbar, ur, uphi, phi], rbar < rbarStar}
  }
 ]
```

In[185]:=

```
Plot[hplusFull[10^6, massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t], phiNumIn[t]],
 {t, 0, tFinish}, PlotRange → Full, Exclusions → None](*10^8 it gets jagged*)
```

Out[185]=



h_x

In[186]:=

```
hxInside[d_, m_, rbar_, ur_, uphi_, phi_] := Module[{r = rofrbarFull[rbar], rd, phid},
  rd = drBardtInside[rbar, ur, uphi];
  phid = dphidtInside[rbar, ur, uphi];
  (-2 m/d)((MassFunc[r] Sin[2 phi]/r - 2 (rd Sin[phi] + r phid Cos[phi]) (rd Cos[phi] - r phid Sin[phi]))
 ]
```

In[187]:=

```
hxOutside[d_, m_, rbar_, ur_, uphi_, phi_] := Module[{r = rofrbarFull[rbar], rd, phid},
  rd = drBardt[rbar, ur, uphi];
  phid = dphidtOutside[rbar, ur, uphi];
  (-2 m/d)((M Sin[2 phi]/r - 2 (rd Sin[phi] + r phid Cos[phi]) (rd Cos[phi] - r phid Sin[phi]))
 ]
```

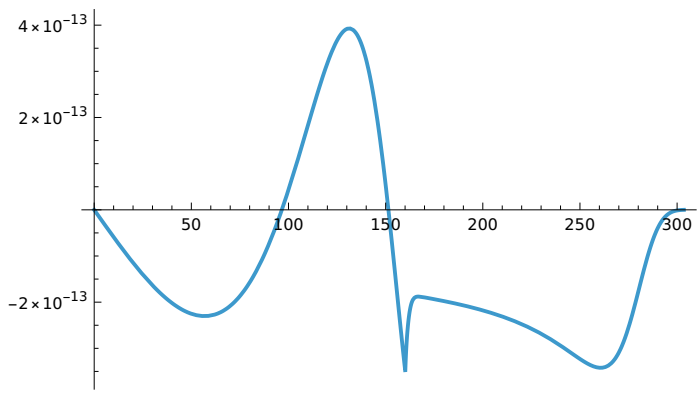
In[188]:=

```
hxFull[d_, m_, rbar_, ur_, uphi_, phi_] := Piecewise[
  {
   {hxOutside[d, m, rbar, ur, uphi, phi], rbar ≥ rbarStar},
   {hxInside[d, m, rbar, ur, uphi, phi], rbar < rbarStar}
  }
 ]
```

In[189]:=

```
Plot[hxFull[10^6, massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t], phiNumIn[t]],
 {t, 0, tFinish}, PlotRange → Full, Exclusions → None](*10^8 it gets jagged*)
```

Out[189]=



Energy loss during orbits **CHECK angles and other assumptions made by authors for these formulae especially the angular momentum stuff.**

Instantaneous energy loss (coordinate system dependent)

In[190]:=

```
dEdtInside[m_, rbar_, ur_, uphi_] :=
 Module[{r = rofrbarFull[rbar], rd, phid}, rd = drBardtInside[rbar, ur, uphi];
  phid = dphidtInside[rbar, ur, uphi];
  (8 m^2)/(15 r^4)*(MassFunction[r]^2*(8 rd^2 + 6 (r phid)^2) + r MassFunction[r]*
      (6 rd^4 - 6 (3 (r phid)^4 + (r phid)^2 Mp[r]) + rd^2 (63 (r phid)^2 + Mp[r] - 3 r Mpp[r])) +
     r^2*(6 (r phid)^4 Mp[r] - rd^2 ((Mp[r])^2 + 3 (r phid)^2 (13 Mp[r] - 3 r Mpp[r])) -
        rd^4 (2 Mp[r] + r Mpp[r] - r^2 Mppp[r])))]
```

In[191]:=

```
dEdtOutside[m_, rbar_, ur_, uphi_] :=
 Module[{r = rofrbarFull[rbar], rd, phid}, rd = drBardt[rbar, ur, uphi];
  phid = dphidtOutside[rbar, ur, uphi];
  (8 m^2)/(15 r^4)*
   (M^2*(8 rd^2 + 6 (r phid)^2) + r M*(6 rd^4 - 6 (3 (r phid)^4) + rd^2 (63 (r phid)^2)))]
```

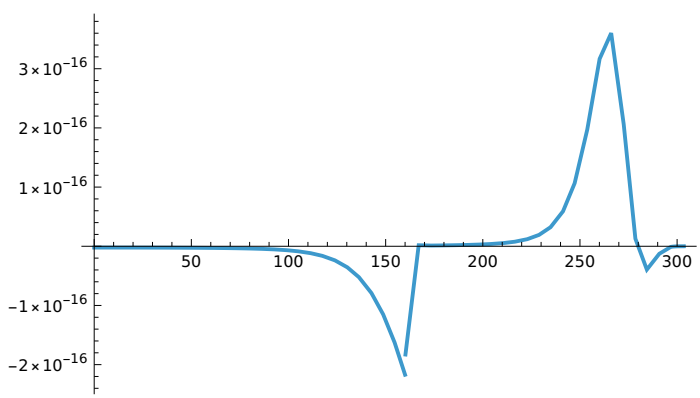
In[192]:=

```
dEdtFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {dEdtOutside[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {dEdtInside[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[193]:=

```
Plot[dEdtFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 0, tFinish}, PlotRange → All]
```

Out[193]=



Average, physical energy loss per orbit

In[194]:=

```
dEdtAvg[m_, M_, a_, e_] := -(32 m^2 M^3)/(5 a^5 (1 - e^2)^(7/2))*(1 + (73/24) e^2 + (37/96) e^4)
```

Need formulae for M[t]? Although accretion is very small. a(t) and e(t)

Angular momentum loss during orbits

Instantaneous angular momentum loss (coordinate system dependent)

In[195]:=

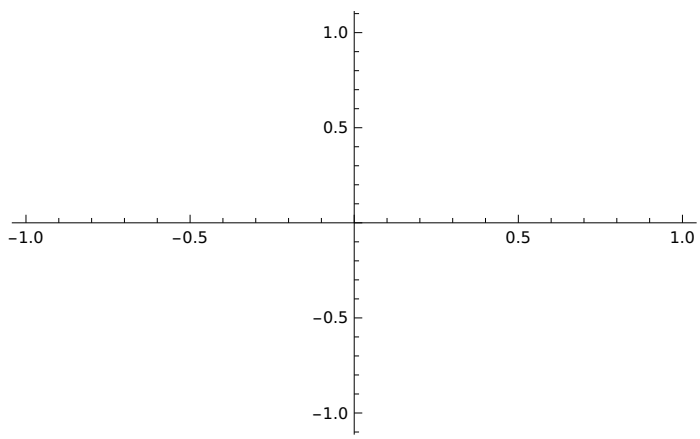
```
dLdt[m_, M_, a_, e_, phi_] :=
```

$$-\frac{4\left(m^2 M^{\frac{5}{2}}\right)}{5\left(a^{\frac{7}{2}}\left(1-e^2\right)^{\frac{7}{2}}\right)}\left(1+e\,\mathrm{Cos}[\mathrm{phi}]\right)^3\left(8-3e^2+20\,e\,\mathrm{Cos}[\mathrm{phi}]+15\,e^2\,\mathrm{Cos}[2\,\mathrm{phi}]\right)$$

In[196]:=

```
Plot[dLdt[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 0, tFinish}, PlotRange → All]
```

Out[196]=



Average, physical angular momentum loss per orbit

In[197]:=

```
(*Formula might need to be adjusted for me as they use the angle phi =
 0 for when it crosses the star surface which is not true for me*)
```

In[198]:=

```
dLdtAvg[m_, M_, a_, e_] := -(32 m ^ 2 M ^ (5 / 2)) / (5 a ^ (7 / 2) (1 - e ^ 2) ^ (2)) * (1 + (7 / 8) e ^ 2)
```

Need formulae for M[t]? Although accretion is very small. a(t) and e(t).

Use E and L to find a(t) and e(t)??